Spectral Sequences for Layered Program Abstraction

Halley Young
Microsoft Research
halleyyoung@microsoft.com

Paper 13 in the Judgment Geometry series March 22, 2026

Abstract

Every non-trivial program possesses a canonical five-level filtration

$$F_0 \subset F_1 \subset F_2 \subset F_3 \subset F_4 = \mathcal{S},$$

corresponding to statements, blocks, functions, modules, and packages. We show that this filtration on the semantic site $\mathcal{S}$ induces a *convergent spectral sequence* $E_r^{p,q} \Rightarrow H^{p+q}(\mathcal{S}, \mathcal{F})$ whose E_2 page has the concrete meaning $E_2^{p,q} = H^p(\text{module-level}, H^q(\text{function-level}, \mathcal{F}))$. Our main result is a *Degeneration Theorem*: the spectral sequence collapses at E_2 if and only if every function is *independently verifiable*—meaning no inter-function obstruction class propagates to higher bidegrees. This structural insight drives a *hierarchical verification algorithm* HierVerify that first computes the E_1 page by verifying each function in isolation, then constructs the E_2 page by checking module-level interface compatibility, and detects unresolved bugs precisely as non-trivial higher differentials $d_r \neq 0$ ($r \geq 2$). All key theorems are formalised in Lean 4 in the companion file `Paper13_SpectralSequences.lean`.

Contents

1 Introduction

A central challenge in program verification is *compositionality*: local proofs about individual functions must be assembled into global guarantees about entire programs. In the JuGeo framework developed across Papers 1–12 of this series, this assembly problem is expressed cohomologically: the obstructions to global verification live in the sheaf cohomology $H^*(\mathcal{S}, \mathcal{F})$ of the semantic site $\mathcal{S}$ with coefficients in the obstruction sheaf $\mathcal{F}$ (Paper 3 [3]). Computing this cohomology directly—as a single monolithic computation—is impractical for large programs.

The classical tool for hierarchically computing cohomology is the *spectral sequence* [10, 9]. Given a filtration $F_0 \subset \cdots \subset F_n$ of a space, the associated spectral sequence provides a page-by-page approximation: the E_1 page sees the local (filtration-graded) picture, the E_2 page incorporates first-order interactions, and successive pages correct higher-order terms until the sequence converges to the actual cohomology at E_∞.

Our contribution. We make three contributions:

1. **The program filtration** (section 3). We identify a canonical five-level filtration of any semantic site,
$$\underbrace{F_0}_{\text{stmts}} \subset \underbrace{F_1}_{\text{blocks}} \subset \underbrace{F_2}_{\text{functions}} \subset \underbrace{F_3}_{\text{modules}} \subset \underbrace{F_4}_{\text{packages}} = \mathcal{S},$$
and establish that this filtration is *exhaustive* and *bounded* in the topos-theoretic sense.

2. **Degeneration at** E_2 (section 7). We prove that the associated spectral sequence degenerates at the E_2 page precisely when each function satisfies the *independent verifiability* condition IndVer: the function's obstruction class has no cross-function extensions.

3. **Hierarchical verification** (section 8). We derive the HierVerify algorithm, which exploits degeneration to reduce global verification to two tractable sub-problems: intra-function analysis (yielding E_1) and inter-module interface checking (yielding E_2). Non-trivial differentials d_r at $r \geq 2$ are interpreted as *cross-boundary bug propagation events*.

Relation to earlier papers. This paper builds directly on the Grothendieck topology framework of Paper 1 [1], the descent obstructions of Paper 3 [3], the trust algebra of Paper 4 [4], and the proof-carrying structure of Paper 9 [7]. The nerve-theoretic lens of Paper 11 [8] provides a simplicial dual picture exploited in section 9.

2 Background: Sheaf Cohomology and Filtered Complexes

We briefly recall the relevant sheaf-theoretic and homological machinery. Throughout, $\mathcal{S} = (\mathcal{C}, J)$ denotes a semantic site: $\mathcal{C}$ is the category of program coordinates and J is the coverage (Paper 1).

2.1 Sheaf cohomology on programs

Let $\mathcal{F}$ be a sheaf of abelian groups on $\mathcal{S}$. In the JuGeo context, $\mathcal{F}$ is typically the *obstruction sheaf* $\mathcal{B}$ of Paper 3, whose sections over an open $U \in \mathcal{C}$ are the obstruction classes blocking verification of the subprogram U. The higher cohomology groups $H^k(\mathcal{S}, \mathcal{F})$ measure the failure of local-to-global passage at degree k.

Definition 2.1 (Čech cohomology, Paper 3)**.** Given a covering $\mathcal{U} = \{U_\alpha\}$ of $\mathcal{S}$, the *Čech complex* $\check{C}^\bullet(\mathcal{U}, \mathcal{F})$ has

$$\check{C}^n(\mathcal{U}, \mathcal{F}) \;=\; \prod_{\alpha_0 < \cdots < \alpha_n} \mathcal{F}(U_{\alpha_0} \cap \cdots \cap U_{\alpha_n}).$$

When $\mathcal{U}$ is acyclic, $\hat{H}^n(\mathcal{U}, \mathcal{F}) \cong H^n(\mathcal{S}, \mathcal{F})$.

2.2 Filtered cochain complexes

Recall that a *filtration* of a cochain complex $(C^\bullet, d)$ is a nested sequence of subcomplexes

$$\cdots \subset F_{p-1}C^\bullet \subset F_pC^\bullet \subset F_{p+1}C^\bullet \subset \cdots \subset C^\bullet.$$

The *graded pieces* are $\mathrm{Gr}^pC^\bullet = F_pC^\bullet / F_{p-1}C^\bullet$. The standard machinery [9, 10] produces a *spectral sequence* $(E_r^{p,q}, d_r)$ with differentials $d_r \colon E_r^{p,q} \to E_r^{p+r,q-r+1}$ and $E_{r+1}^{p,q} = H(E_r^{p,q}, d_r)$.

Definition 2.2 (Convergence)**.** The spectral sequence *converges* to $H^{p+q}(C^\bullet)$, written $E_r^{p,q} \Rightarrow H^{p+q}(C^\bullet)$, if for each total degree n there is a filtration of $H^n(C^\bullet)$ whose graded pieces are the $E_\infty^{p,n-p}$.

Definition 2.3 (Degeneration at E_r)**.** A spectral sequence *degenerates at* E_r if $d'_{\lfloor} = 0$ for all $r' \geq r$, so that $E_r^{p,q} \cong E_\infty^{p,q}$ for all p, q.

3 The Program Filtration

3.1 The five-level hierarchy

Every non-trivial program's semantic site $\mathcal{S}$ admits a canonical *lexicographic filtration* by abstraction level.

Definition 3.1 (Program filtration)**.** Let $\mathcal{S}$ be the semantic site of a program. The *canonical filtration* $\mathcal{S}_\bullet$ is the chain of subsites

$$F_0 \subset F_1 \subset F_2 \subset F_3 \subset F_4 = \mathcal{S}$$

where F_p consists of all coordinates of abstraction level $\leq p$:

$$\begin{aligned} F_0 &= \{\, c \in \mathcal{S} \mid \mathrm{level}(c) = \mathsf{stmt} \,\}, \\ F_1 &= \{\, c \in \mathcal{S} \mid \mathrm{level}(c) \leq \mathsf{blk} \,\}, \\ F_2 &= \{\, c \in \mathcal{S} \mid \mathrm{level}(c) \leq \mathsf{fun} \,\}, \\ F_3 &= \{\, c \in \mathcal{S} \mid \mathrm{level}(c) \leq \mathsf{mod} \,\}, \\ F_4 &= \mathcal{S}. \end{aligned}$$

The graded pieces $\mathrm{Gr}^p\mathcal{S} = F_p/F_{p-1}$ isolate the coordinates first appearing at abstraction level p: for instance, $\mathrm{Gr}^2\mathcal{S}$ contains exactly the function-level coordinates modulo their block-level substructure.

3.2 Properties of the filtration

Lemma 3.2 (Strict chain)**.** *The inclusions are strict: $F_{p-1} \subsetneq F_p$ for $p = 1, 2, 3, 4$, provided each abstraction level is non-empty.*

Proof. Any coordinate of level exactly p belongs to F_p but not to F_{p-1}, since F_{p-1} contains only coordinates of level $\leq p-1$. □

Lemma 3.3 (Exhaustiveness)**.** $\bigcup_{p=0}^{4} F_p = \mathcal{S}$.

Proof. Every coordinate $c \in \mathcal{S}$ has $\mathrm{level}(c) \leq 4$ by definition, hence $c \in F_4 = \mathcal{S}$. □

Lemma 3.4 (Monotone restriction)**.** *If $p \leq q$, then $F_p \subset F_q$ and restriction maps $\mathrm{res}_p^q \colon \mathcal{F}(F_q) \to \mathcal{F}(F_p)$ are well-defined.*

Proof. Since $F_p \subset F_q$, any section over F_q restricts canonically to F_p via the sheaf restriction maps. □

3.3 Filtered cochain complex of the semantic site

Given the program filtration $\mathcal{S}_\bullet$ and a sheaf $\mathcal{F}$ on $\mathcal{S}$, we obtain a *filtered cochain complex* $(\check{C}^\bullet(\mathcal{S}, \mathcal{F}), d, F_\bullet)$ by setting

$$F_p\check{C}^n(\mathcal{S}, \mathcal{F}) \;=\; \ker\big(\check{C}^n(\mathcal{S}, \mathcal{F}) \to \check{C}^n(F_{p-1}, \mathcal{F})\big).$$

Proposition 3.5. *The filtration $F_\bullet\check{C}^\bullet$ is compatible with the Čech differential d: $d(F_p\check{C}^n) \subset F_p\check{C}^{n+1}$. Hence $(F_\bullet\check{C}^\bullet, d)$ is a filtered cochain complex.*

Proof. The Čech differential maps a cochain supported on level-$\geq p$ coordinates to a cochain still supported on level-$\geq p$ coordinates, since the coboundary formula involves only intersection terms within the same abstraction level. □

4 The Associated Spectral Sequence

4.1 Construction

Applying the spectral sequence machine to $(F_\bullet \check{C}^\bullet, d)$:

Definition 4.1 (Program spectral sequence)**.** The *program spectral sequence* is

$$E_1^{p,q} \;=\; H^{p+q}\big(\mathrm{Gr}^p \check{C}^\bullet\big) \;=\; H^{p+q}\big(F_p\mathcal{S},\, F_{p-1}\mathcal{S};\, \mathcal{F}\big),$$

with differentials $d_r \colon E_r^{p,q} \to E_r^{p+r,q-r+1}$ and $E_{r+1}^{p,q} = H(E_r^{p,q}, d_r)$.

The differential d_r moves r steps to the right in p and $r-1$ steps down in q.

4.2 Convergence

Theorem 4.2 (Convergence of the program spectral sequence)**.** *The program spectral sequence converges:*

$$E_1^{p,q} \;\Rightarrow\; H^{p+q}(\mathcal{S}, \mathcal{F}).$$

There is a filtration $F_pH^n(\mathcal{S},\mathcal{F})$ such that $E_\infty^{p,q} \cong F_pH^{p+q}/F_{p-1}H^{p+q}$.

Proof. The filtration is bounded ($0 \le p \le 4$) and exhaustive (theorem 3.3). For a bounded exhaustive filtration, the Eilenberg–Moore comparison theorem [9] §5.4 guarantees convergence. □

4.3 The E_1 and E_2 pages

Proposition 4.3 (E_1 page)**.**

$$E_1^{p,q} \;=\; H^{p+q}\big(F_p,\, F_{p-1};\, \mathcal{F}\big) \;\cong\; \bigoplus_{c \,\in\, \mathrm{Gr}^p\mathcal{S}} H^q(c, \mathcal{F}|_c).$$

The differential $d_1 \colon E_1^{p,q} \to E_1^{p+1,q}$ is the module-interface map*: it computes the obstruction to gluing a function-level section across the boundary between levels p and $p+1$.*

Proof. The relative cohomology $H^{p+q}(F_p, F_{p-1}; \mathcal{F})$ decomposes as a direct sum over coordinates first appearing at level p by excision. □

Proposition 4.4 (E_2 page)**.**

$$E_2^{p,q} \;=\; H^p\big(\textit{level-}p\textit{ skeleton},\, H^q(\textit{function-level}, \mathcal{F})\big).$$

In particular, $E_2^{2,0}$ consists of function-level sections compatible across all module interfaces, and $E_2^{3,0}$ consists of module-level sections compatible across all package-level interfaces.

The E_2 page records *cohomology of cohomology*: first resolve within each function (getting $E_1^{*,q}$), then check compatibility at the module level.

q					
2	$E_1^{0,2}$	$E_1^{1,2}$	$E_1^{2,2}$	$E_1^{3,2}$	$E_1^{4,2}$
1	$E_1^{0,1}$	$E_1^{1,1}$	$E_1^{2,1}$	$E_1^{3,1}$	$E_1^{4,1}$
0	$E_1^{0,0}$	$E_1^{1,0}$	$E_1^{2,0}$	$E_1^{3,0}$	$E_1^{4,0}$
	p=0	p=1	p=2	p=3	p=4
	stmt	blk	fun	mod	pkg

Figure 1: The E_1 page. The differential d_1 points horizontally: $E_1^{p,q} \to E_1^{p+1,q}$. Taking cohomology horizontally yields E_2.

5 The E_1 Page: Function-Level Verification

The E_1 page is the natural output of *per-function verification*. For each function coordinate $f \in \mathrm{Gr}^2\mathcal{S}$:

$$E_1^{2,q} \;=\; \bigoplus_{f \in \mathrm{Gr}^2\mathcal{S}} H^q(f, \mathcal{F}|_f).$$

Definition 5.1 (Function-level obstruction class)**.** A *function-level obstruction class* is an element of $E_1^{2,0} = \bigoplus_f \mathcal{F}(f)$, corresponding to a judgment $\mathcal{J} = (\mathsf{c},\, \phi,\, \mathcal{A},\, \mathcal{E},\, \mathcal{O},\, \mathcal{B},\, \mathcal{T},\, \Pi)$ with $\mathsf{c} \in \mathrm{Gr}^2\mathcal{S}$ and $\mathcal{B} \neq 0$.

Example 5.2 (Off-by-one bug)**.** A function `find_last` uses index $i \leq \text{len}$ instead of $i < \text{len}$. The obstruction $b \in E_1^{2,0}$ is local: it lives at $p = 2$ and $d_1(b) = 0$ because b does not extend to a module-level obstruction. It is independently fixable.

Example 5.3 (Cross-function type mismatch)**.** Function `parse` returns `Token?` (nullable) while caller `eval` assumes non-nullable `Token`. The class $b \in E_1^{2,0}$ for `parse` has $d_1(b) \neq 0$: the obstruction propagates to level $p = 3$, contributing a class to $E_1^{3,0}$. This signals a *cross-function* bug.

5.1 Computing E_1 in practice

Computing E_1 reduces to independent judgment-verification problems, one per function. In the JuGeo framework this corresponds to running the SMT dispatch layer (Paper 5 [5]) for each function in isolation. The trust tier of each E_1 class reflects the verification method: $\mathcal{T}_f = \mathsf{SOLVER}$ if discharged by an SMT solver; $\mathcal{T}_f = \mathsf{PROOF}$ if by a mechanised proof.

6 The E_2 Page: Module-Level Composition

The E_2 page arises by taking cohomology of the horizontal differential d_1:

$$E_2^{p,q} \;=\; \ker\bigl(d_1 \colon E_1^{p,q} \to E_1^{p+1,q}\bigr) \,/\, \mathrm{im}\bigl(d_1 \colon E_1^{p-1,q} \to E_1^{p,q}\bigr).$$

Definition 6.1 (Module-interface compatibility)**.** Functions $f, g \in \mathrm{Gr}^2\mathcal{S}$ in the same module $M \in \mathrm{Gr}^3\mathcal{S}$ are *interface-compatible* if $d_1(\mathcal{B}_f) = 0 \in E_1^{3,0}$.

Proposition 6.2 (E_2 and treaty verification)**.** *The term $E_2^{3,0}$ is isomorphic to the space of treaty-compatible module sections (Paper 8 [6]): module-level sections in the kernel of d_1 and not in the image of d_1 from function level.*

Proof. By definition, $E_2^{3,0} = \ker(d_1 \colon E_1^{3,0} \to E_1^{4,0})/\mathrm{im}(d_1 \colon E_1^{2,0} \to E_1^{3,0})$. The kernel condition is package-compatibility; the image condition removes classes already accounted for by function-level repairs. This recovers the treaty-compatible section definition of Paper 8. □

Remark 6.3 (Trust inheritance at E_2). If each function-level class carries trust $\geq$ SOLVER, then the d_1-boundary computation is a purely syntactic interface check dischargeable by `QF_LIA` or `QF_UF` (Paper 5 [5]), so E_2 classes inherit trust $\geq$ SOLVER.

7 The Degeneration Theorem

7.1 Independent verifiability

Definition 7.1 (Independently verifiable function)**.** A function $f \in \mathrm{Gr}^2\mathcal{S}$ is *independently verifiable* ($\mathsf{IndVer}(f)$) if for every module $M \ni f$ and every other function $g \in M$, the extension map $\mathrm{ext}_{f,g} \colon \mathcal{F}(f) \to \mathcal{F}(f \cup g)$ is the zero map.

$\mathsf{IndVer}(f)$ means the obstruction class of f does not "leak" into joint obstructions with other functions. This holds whenever f's specification boundary is self-contained.

Lemma 7.2 (Independent verifiability and d_1)**.** *If $\mathsf{IndVer}(f)$ holds for every $f \in \mathrm{Gr}^2\mathcal{S}$, then $d_1 \colon E_1^{2,0} \to E_1^{3,0}$ is the zero map.*

Proof. The map $d_1|_{p=2}$ is built from the coboundary maps $\mathcal{F}(f) \to \mathcal{F}(f \cup g)$. Under $\mathsf{IndVer}(f)$ each such map is zero, so the total differential vanishes. □

7.2 Main theorem

Theorem 7.3 (Degeneration at E_2)**.** *The program spectral sequence $(E_r^{p,q}, d_r)$ degenerates at E_2—i.e. $d_r = 0$ for all $r \geq 2$—if and only if every function in $\mathrm{Gr}^2\mathcal{S}$ is independently verifiable.*

Proof. ($\Leftarrow$) Assume $\mathsf{IndVer}(f)$ for all f. By theorem 7.2, $d_1|_{p=2} = 0$. For $d_2 \colon E_2^{p,q} \to E_2^{p+2,q-1}$: the zig-zag definition of d_2 requires a lift through the filtration of a d_1-closed element. Under independent verifiability, any coboundary of $x \in F_2C^*$ that is d_1-closed already lands in F_2C^*. The resulting d_2-class is killed by the independence condition at the previous stage. By induction on r, every higher differential vanishes.

($\Rightarrow$) If $\mathsf{IndVer}(f)$ fails for some f, there exist f, g in the same module with $\mathrm{ext}_{f,g}(b) \neq 0$. Then $d_1(b) \neq 0$, and a standard acyclicity argument shows the spectral sequence does not degenerate at E_2. □

Corollary 7.4 (Proof reusability)**.** *Under degeneration at E_2:*

$$H^n(\mathcal{S}, \mathcal{F}) \;\cong\; \bigoplus_{p+q=n} E_2^{p,q}.$$

Fixing a bug in function f does not affect any other E_2 terms; proofs are fully reusable across functions.

Proof. Under degeneration $E_\infty^{p,q} = E_2^{p,q}$. When each $E_2^{p,q}$ is projective, the filtration on H^n splits as a direct sum. □

Remark 7.5 (Trust monotonicity under degeneration). Under degeneration, the trust tier of each total cohomology class is the meet of the contributing function-level trust tiers. No trust attenuation occurs from higher differentials, since they are zero. Trust assignment is *additive over functions.*

8 Hierarchical Verification Algorithm

8.1 The HierVerify algorithm

Construction 8.1 (HierVerify)**.** Given a program with semantic site $\mathcal{S}$ and obstruction sheaf $\mathcal{F}$:

Phase 1: Function-level verification (E_1 computation). For each function $f \in \mathrm{Gr}^2\mathcal{S}$:

(i) Run the SMT dispatch (Paper 5) on f's pre/postcondition pair.

(ii) Record $\mathcal{B}_f = 0$ (success) or $b_f \in E_1^{2,0}$ (failure).

Phase 2: Interface compatibility (E_2 computation). For each module M and each adjacent pair $(f, g) \in M$:

(i) Compute $d_1(b_f)$. If non-zero, record a module-level bug in $E_1^{3,0}$.

(ii) Compute $E_2^{p,q} = H(E_1^{p,q}, d_1)$ for all (p, q).

Phase 3: Higher differential check.

(i) If $d_2 = 0$: output *degeneration holds*, $H^*(\mathcal{S}, \mathcal{F}) \cong \bigoplus_{p,q} E_2^{p,q}$.

(ii) If $d_2(x) \neq 0$: output *package-level obstruction*—a cross-module bug propagating across the package boundary.

8.2 Complexity analysis

Proposition 8.2 (Complexity)**.** *Let N_f be the number of functions, N_m the number of modules, and N_{iface} the total interface pairs.*

- *Phase 1 costs $O(N_f \cdot T_{\text{SMT}})$, embarrassingly parallel.*
- *Phase 2 costs $O(N_{\text{iface}})$ interface checks.*
- *Phase 3 costs $O(N_m^2)$ differential computations.*

Under degeneration, Phase 3 terminates in one step.

8.3 Bug propagation via higher differentials

- $d_1(b) \neq 0$: bug b propagates from function f to sibling g in the same module. *Fix*: strengthen the shared interface.
- $d_2(b) \neq 0$: bug b survives module-level compatibility but propagates across a package boundary. *Fix*: add a package-level treaty (Paper 8 [6]).
- $d_r(b) \neq 0$ $(r \geq 3)$: bug involves three or more nested abstraction levels. *Fix*: refactor the abstraction hierarchy.

9 Illustrative Examples

9.1 Clean module (degeneration holds)

Module `math_utils` contains `add`, `sub`, `mul`, each with self-contained pre/postconditions. All $b_f = 0$, so $d_1 = d_2 = 0$. The sequence degenerates at $E_1 = E_2 = E_\infty$ and $H^*(\texttt{math\_utils}, \mathcal{F}) = 0$: the module is verified.

9.2 Isolated bug

Suppose `sub` has an off-by-one: $b_{\texttt{sub}} \neq 0$. If $\mathsf{IndVer}(\texttt{sub})$ holds, then $d_1(b_{\texttt{sub}}) = 0$. The E_2 page contains $E_2^{2,0} \ni [b_{\texttt{sub}}]$ and $d_2 = 0$. Total cohomology: $H^0 \cong \langle b_{\texttt{sub}} \rangle$—a single isolated bug, fixable without touching other functions.

9.3 Cross-function bug (non-degeneration)

Suppose `mul` relies on `sub`'s postcondition P that `sub` fails to establish. Then $d_1(b_{\texttt{sub}}) = c \neq 0$, and $E_2^{3,0} \ni c$. If the module is self-contained, $d_2(c) = 0$ and the sequence degenerates at E_2: the bug is a *module-level interaction* between `sub` and `mul`, correctable by strengthening the shared interface treaty.

9.4 Spectral sequence grid

q					
1	0	0	$\langle b' \rangle$	0	0
0	0	0	$\langle b_{\texttt{sub}} \rangle$	$\langle c \rangle$	0
	stmt	blk	fun	mod	pkg

Figure 2: E_1 page for the cross-function example. The differential d_1 sends $b_{\texttt{sub}} \mapsto c$. After taking cohomology, $E_2^{2,0} = 0$ and $E_2^{3,0} \cong \langle c \rangle$.

9.5 Connection to nerve theorems (Paper 11)

The Nerve Lemma of Paper 11 [8] provides a simplicial dual to the spectral sequence computation. The E_1 page corresponds to the 0-skeleton of the nerve, and d_1 computes the 1-skeleton boundary maps. Degeneration at E_2 corresponds to the nerve being 1-dimensional (a forest):

Proposition 9.1 (Degeneration and nerve dimension)**.** *The program spectral sequence degenerates at E_2 if and only if the nerve $\mathcal{N}(\mathcal{U}_{\mathsf{fun}})$ of the function-level covering has dimension ≤ 1 (i.e. is homotopy equivalent to a forest).*

10 Discussion and Related Work

Classical spectral sequences. The construction is a concrete instance of the Leray–Serre spectral sequence [11, 12] applied to the map $\pi \colon \mathcal{S} \to B$ where $B = \{0, 1, 2, 3, 4\}$ and the fibers are the graded pieces. Our program-filtration context gives each level a concrete syntactic meaning, which is non-standard.

Verification hierarchies. Compositional verification has a long history: Hoare's compositional rules [13], modular verification in VCC [14], and separation logic [15]. Our contribution is to *quantify* the composition cost via spectral sequence differentials and give a precise degeneration criterion.

Descent and sheaves. The use of sheaf theory in program verification was established in Papers 1–3 of this series. Spectral sequences have not, to our knowledge, been previously applied to program verification.

11 Conclusion

We have shown that the canonical five-level program filtration induces a convergent spectral sequence $E_r^{p,q} \Rightarrow H^{p+q}(\mathcal{S}, \mathcal{F})$ whose E_2 page has the concrete interpretation $E_2^{p,q} = H^p(\text{module-level}, H^q(\text{function-leve}$ The Degeneration Theorem characterises the collapse at E_2 as precisely independent verifiability of all functions.

The HierVerify algorithm reduces global verification to independent per-function SMT queries followed by module-level interface checks, with higher differentials flagging cross-boundary bug propagation. Under degeneration, trust assignments are additive over functions (no attenuation from higher differentials).

Future directions: (i) extending the filtration to cyclic module dependencies via the Eilenberg–Moore spectral sequence; (ii) automating Phase 3 of HierVerify by encoding d_2 checks as `QF_UF` queries; (iii) connecting degeneration to the bidirectional certificates of Paper 9 [7].

References

[1] H. Young. Grothendieck Topologies for Semantic Program Verification. *Judgment Geometry Series, Paper 1*, 2024.

[2] H. Young. Judgment Algebra: A Compositional Framework for Program Verification. *Judgment Geometry Series, Paper 2*, 2024.

[3] H. Young. Descent Obstructions in Sheaf-Theoretic Program Verification. *Judgment Geometry Series, Paper 3*, 2024.

[4] H. Young. Trust Algebra for Multi-Source Evidence. *Judgment Geometry Series, Paper 4*, 2024.

[5] H. Young. SMT Fragment Dispatch in Sheaf-Theoretic Verification. *Judgment Geometry Series, Paper 5*, 2024.

[6] H. Young. Treaty Synthesis for Interface Reconciliation. *Judgment Geometry Series, Paper 8*, 2024.

[7] H. Young. Proof-Carrying Python: Certificates in the Semantic Site. *Judgment Geometry Series, Paper 9*, 2024.

[8] H. Young. Nerve Theorems for Code Coverage Completeness. *Judgment Geometry Series, Paper 11*, 2024.

[9] C. A. Weibel. *An Introduction to Homological Algebra*. Cambridge University Press, 1994.

[10] J. McCleary. *A User's Guide to Spectral Sequences*. Cambridge University Press, 2nd ed., 2001.

[11] J. Leray. Structure de l'anneau d'homologie d'une représentation. *C. R. Acad. Sci. Paris*, 222:1419–1422, 1946.

[12] J.-P. Serre. Homologie singulière des espaces fibrés. *Ann. Math.*, 54(3):425–505, 1951.

[13] C. A. R. Hoare. An axiomatic basis for computer programming. *Commun. ACM*, 12(10):576–580, 1969.

[14] E. Cohen et al. VCC: A practical system for verifying concurrent C. In *TPHOLs 2009*, LNCS 5674, pp. 23–42, 2009.

[15] J. C. Reynolds. Separation logic: A logic for shared mutable data structures. In *LICS 2002*, pp. 55–74, 2002.